MODULE

SESSION 09

Subqueries and CTEs, then views

How do I break a hard question into named steps, and save them for Power BI?

Tarek Atwan · Lead Instructor

Sun 11 Oct · 4:00–8:00 PM

DATA SCIENCE & ANALYTICS

BY THE END OF TODAY, YOU CAN

Class objectives

01 Put a subquery in WHERE, SELECT or FROM

02 Rewrite a nested query as a CTE

03 Plan a question as named steps

04 Join two CTEs to compare two years

05 Save a CTE as a view

EVERYONE DO Step 1.1

Warm-up

Four questions on Wednesday’s subqueries, and one on how you feel about them.

Suggested time · 10 minutes

PART 02

Subqueries: a review

One inner query, moved between three places. Where it goes decides what it must return.

INSTRUCTOR DO Step 2.1

Three places a subquery can live

“The average completed order”, in WHERE, SELECT and FROM.

Suggested time · 13 minutes

`labs/D09/01-Ins-Three-Places/README.md`

DEFINITION

A subquery is a query inside another query, in brackets. The inner one runs first; the outer one uses its answer.

STEP 1 · THE INNER QUERY ALONE

The average completed order

One row is one ___? Predict, then run.

```
SELECT AVG(order_total) AS average_order
FROM nakheel.orders
WHERE status = 'completed';



                                                → 1 row: 114.05
```

STEP 2 · IN WHERE

One value to compare with

`>` compares every order with one number.

```
SELECT order_id, customer_id, order_total
FROM nakheel.orders
WHERE status = 'completed'
  AND order_total > (SELECT AVG(order_total)
                     FROM nakheel.orders
                     WHERE status = 'completed')
ORDER BY order_total DESC;
```

```
→ 689 rows · order 100849 first, 1,964.50
```

STEP 3 · IN SELECT · SHOWN, NOT TYPED

Exactly one value, on every row

A subquery in SELECT fills one cell of every row.

```
SELECT order_id,
       order_total,
       (SELECT ROUND(AVG(order_total), 2)
        FROM nakheel.orders
        WHERE status = 'completed') AS average_order
FROM nakheel.orders
WHERE status = 'completed';
```

```
→ 2,771 rows · 114.05 on every row
```

STEP 4 · THE INNER QUERY FOR FROM

First, one row per customer

One row is one ___? Predict the row count.

```sql
SELECT customer_id,
       SUM(order_total) AS revenue
FROM nakheel.orders
WHERE status = 'completed'
GROUP BY customer_id;

                                                  → 452 rows
```

STEP 4 · IN FROM

A table, with a name

`FROM ( … ) AS per_customer` is read like a table.

```
SELECT COUNT(*) AS customers_over_1000
FROM (SELECT customer_id,
             SUM(order_total) AS revenue
      FROM nakheel.orders
      WHERE status = 'completed'
      GROUP BY customer_id) AS per_customer
WHERE revenue > 1000;
```

```
→ 1 row: 60 customers
```

THREE PLACES

Where the brackets go decides what they return

Place	The outer query needs	Today’s example
WHERE	one value, or a list with IN	orders above the average
SELECT	exactly one value, on every row	each order next to the average
FROM	a table, named with AS	customers who spent over 1,000

READ IT INSIDE OUT

A nested query runs from the inside out. People read from the top. A CTE closes that gap: after 3.1, the steps read in the order they run.

EVERYONE DO Step 2.2

Do our customers come back?

“How many customers bought once, and how many came back?” One row per order in; one row per customer needed.

Suggested time · 12 minutes

`labs/D09/02-Evr-Subquery-In-From/README.md`

QUERY 1 · STEP 1 ALONE

Completed orders per customer

One row is one ___? Predict the row count.

```sql
SELECT customer_id,
       COUNT(*) AS completed_orders
FROM nakheel.orders
WHERE status = 'completed'
GROUP BY customer_id
ORDER BY completed_orders DESC;
```

```
→ 452 rows · C-1270 first, 126
```

PLANNED DEAD END

Two grains in one SELECT

`AVG(COUNT(*))` asks for two grains at once.

```
SELECT AVG(COUNT(*)) AS avg_orders
FROM nakheel.orders
WHERE status = 'completed'
GROUP BY customer_id;
```

```
Multi-level aggregation requires the enclosing aggregate function to have one
or more GROUP BY modifiers
```

Step 1 has to finish first: put it in FROM.

QUERY 2 · STEP 1 IN FROM

The average customer, the fewest and the most

The outer query only sees the columns step 1 kept.

```
SELECT ROUND(AVG(completed_orders), 2) AS avg_orders,
       MIN(completed_orders) AS fewest,
       MAX(completed_orders) AS most
FROM (SELECT customer_id,
             COUNT(*) AS completed_orders
      FROM nakheel.orders
      WHERE status = 'completed'
      GROUP BY customer_id) AS per_customer;

→ 1 row: 6.13 · 1 · 126
```

QUERY 3 · THE MANAGER’S QUESTION

Bought once, or came back?

A CASE in the outer query labels each customer.

```
SELECT CASE
         WHEN completed_orders = 1 THEN 'one order'
         ELSE 'came back'
       END AS kind,
       COUNT(*) AS customers
FROM (SELECT customer_id,
             COUNT(*) AS completed_orders
      FROM nakheel.orders
      WHERE status = 'completed'
      GROUP BY customer_id) AS per_customer
GROUP BY kind
ORDER BY kind;
```

```
                                              → 2 rows · came back 323 · one order 129
```

PART 03

CTEs: name each step

The same steps, written top to bottom with names.

INSTRUCTOR DO Step 3.1

From nested to named

“What does a typical customer who has bought from us spend, in each segment?”

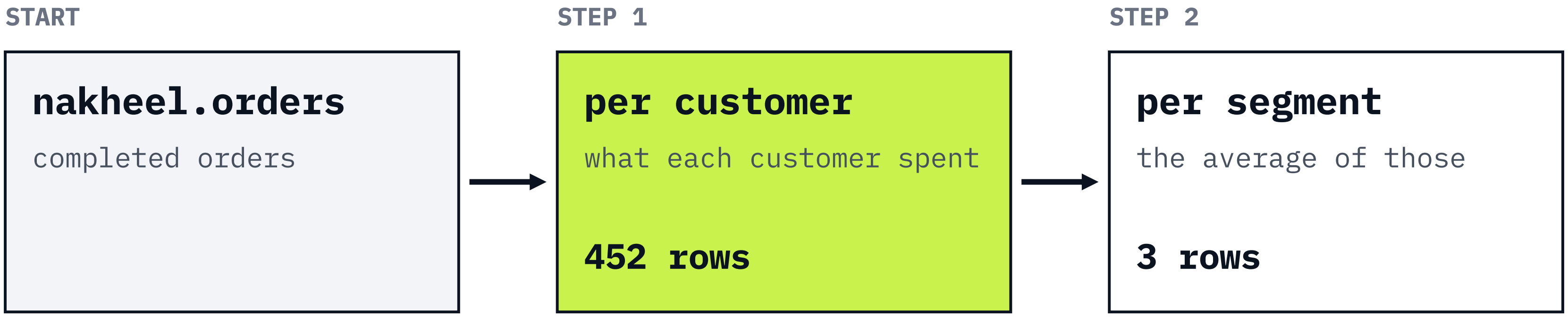
Suggested time · 12 minutes

`labs/D08/04-Ins-Nested-To-Named/README.md`

TWO GRAINS

One GROUP BY cannot answer it

First one row per customer: what each spent. Then one row per segment: the average of those.



START

nakheel.orders

completed orders

STEP 1

per customer

what each customer spent

452 rows

STEP 2

per segment

the average of those

3 rows

STEP 1 · ONE ROW PER CUSTOMER

What each customer spent

One row is one ___? Predict the row count.

```
SELECT customer_id,
       COUNT(*) AS completed_orders,
       SUM(order_total) AS revenue
FROM nakheel.orders
WHERE status = 'completed'
GROUP BY customer_id;
```

```
→ 452 rows
```

NESTED · PASTE IT

Step 1 in brackets, inside step 2

It starts with the answer’s columns, then dives into brackets for the part that runs first.

```
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,
       COUNT(*) AS customers,
       ROUND(AVG(cr.revenue), 2) AS avg_revenue_per_customer
FROM (SELECT customer_id, COUNT(*) AS completed_orders, SUM(order_total) AS revenue
      FROM nakheel.orders
      WHERE status = 'completed'
      GROUP BY customer_id) AS cr
LEFT JOIN nakheel.customers AS c
  ON cr.customer_id = c.customer_id
GROUP BY segment_clean
ORDER BY avg_revenue_per_customer DESC;
```

→ 3 rows

DEFINITION

A CTE (common table expression) is a named step at the top of a query. The SELECT after it reads the name like a table.

CTE · 1 · NAME THE STEP

WITH gives step 1 a name

`WITH customer_revenue AS ( … )` runs step 1 and names its result.

```
WITH customer_revenue AS (
  SELECT customer_id,
         COUNT(*) AS completed_orders,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY customer_id
)
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,
       COUNT(*) AS customers,
       ROUND(AVG(cr.revenue), 2) AS avg_revenue_per_customer
FROM customer_revenue AS cr
LEFT JOIN nakheel.customers AS c
  ON cr.customer_id = c.customer_id
GROUP BY segment_clean
ORDER BY avg_revenue_per_customer DESC;
```

CTE · 2 · USE THE NAME

The SELECT reads the name like a table

Top to bottom, in the order it runs.

```
WITH customer_revenue AS (
  SELECT customer_id,
         COUNT(*) AS completed_orders,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY customer_id
)
SELECT COALESCE(LOWER(TRIM(c.segment)), 'unknown') AS segment_clean,
       COUNT(*) AS customers,
       ROUND(AVG(cr.revenue), 2) AS avg_revenue_per_customer
FROM customer_revenue AS cr
LEFT JOIN nakheel.customers AS c
  ON cr.customer_id = c.customer_id
GROUP BY segment_clean
ORDER BY avg_revenue_per_customer DESC;
```

THE ANSWER · 3 ROWS

Wholesale: about six and a half times retail

The D07 totals hid this: wholesale had fewer customers but nearly half the revenue.

segment_clean	customers	avg_revenue_per_customer
wholesale	58	**2,658.69**
unknown	1	938.00
retail	393	**409.37**

EVERYONE DO Step 3.2

A two-step pipeline

“Which months were better than our average month?” Plan the steps aloud before typing.

Suggested time · 15 minutes

`labs/D08/05-Evr-Two-CTE-Pipeline/README.md`

PLAN THE STEPS FIRST

Three steps, each with a name

Each step reads the one before it. Check its rows before adding the next.

STEP 1

monthly

revenue per month

24 rows

→

STEP 2

average_month

the average of those months

1 row

→

STEP 3

the answer

the months above it

8 rows

STEP 1 · monthly

Revenue per month, checked on its own

SELECT * FROM monthly shows step 1 before anything reads it.

```
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS month,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY month
)
SELECT *
FROM monthly
ORDER BY month;
                               → 24 rows · September 2024 first, 11,268.00
```

STEP 2 · average_month

A second step reads the first

After a comma, the second CTE. Check it alone.

```sql
average_month AS (
  SELECT AVG(revenue) AS avg_revenue
  FROM monthly
)
```

```
→ 1 row: 13,167.65
```

STEP 3 · THE ANSWER

One number, used as a subquery

Step 2 is one number, so > can use it: the 2.1 idea again.

```
SELECT month,
       ROUND(revenue, 2) AS revenue
FROM monthly
WHERE revenue > (SELECT avg_revenue FROM average_month)
ORDER BY month;
```

→ 8 rows · April 2026 the best, 20,237.00

STUDENT DO

Activity: One row per customer

In this activity, you will build a customer summary in two named steps, for every customer, and audit it.

Suggested time

18 minutes

`labs/D08/06-Stu-Customer-Summary/README.md`

STUDENT DO · 18 MINUTES

Activity: One row per customer

INSTRUCTIONS	1. Per customer: completed orders, revenue, first and last completed order date. 2. Every customer, including those with no completed order (0, not blank). 3. Audit: customers, orders, revenue, and how many have none.
HINT	Step 1 is one row per customer with a completed order. Step 2 keeps every customer.
NEW	COUNTIF counts the rows where a test is true. It is a BigQuery function.

`Full steps and Check yourself: labs/D08/06-Stu-Customer-Summary/README.md`

REVIEW

Time's up.
Let's review.

One row per customer

`Solved: labs/D08/06-Stu-Customer-Summary/Solved/customer_summary.sql`

THE AUDIT · QUESTION 3

Why 315,085.50 and not 316,023.50?

The summary: 500 customers, 2,770 orders, 49 with none.

From customers

315,085.50

On orders

316,023.50

938.00 is order 101664: its customer is not in customers, so it has no row here.

PARTNER DO

Activity: Plan the steps, then write them

In this activity, you and a partner will answer two manager questions as CTEs, planning the steps in comments before any SQL.

Suggested time

20 minutes

`labs/D09/03-Par-Plan-The-Steps/README.md`

PARTNER DO · 20 MINUTES

Activity: Plan the steps, then write them

INSTRUCTIONS	1. Which categories brought in more than the average category? 2. How many customers spent more than the average customer, and what share of revenue do they bring? Plan each step in a comment first. Check its rows before the next.
HINT	Divide by the total of step 1, not a number you type.
BONUS	Which segment are the big customers in?

Full steps and Check yourself: `labs/D09/03-Par-Plan-The-Steps/README.md`

REVIEW

Time's up.
Let's review.

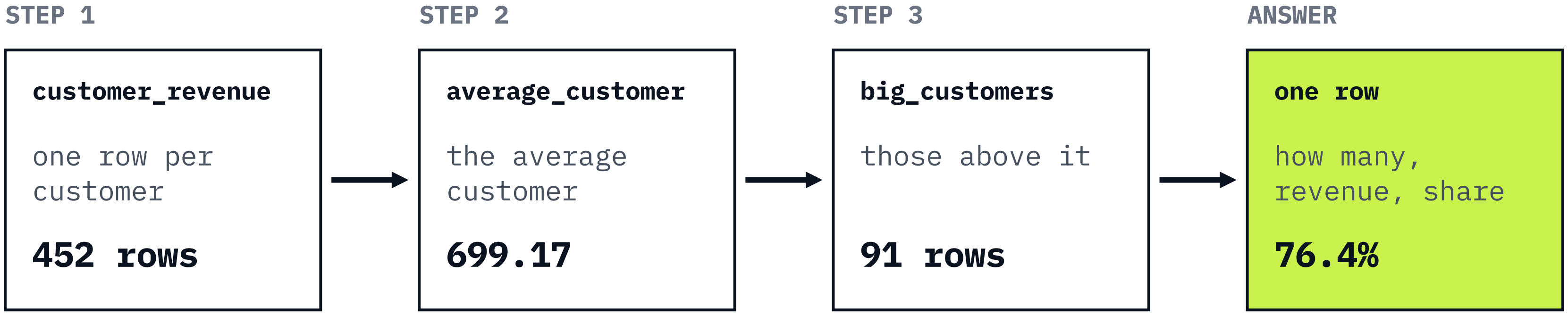
Plan the steps

Solved: `labs/D09/03-Par-Plan-The-Steps/Solved/plan_the_steps.sql`

QUESTION 2 · FOUR NAMED STEPS

Each step reads the one before it

Name each step after what one row of it is. Check its rows before the next.



STEP 1

`customer_revenue`

one row per customer

452 rows

STEP 2

`average_customer`

the average customer

699.17

STEP 3

`big_customers`

those above it

91 rows

ANSWER

one row

how many, revenue, share

76.4%

QUESTION 2 · THE ANSWER

76.4%

of completed revenue comes from 91 customers, one in five of the 452 who bought.

labs/D09/03-Par-Plan-The-Steps · 241,338.00 of 316,023.50

EVERYONE DO Step 3.7

Untangle a nested query

Two nested queries that work, rewritten as CTEs. Same answer, read top to bottom.

Suggested time · 15 minutes

`labs/D09/04-Evr-Untangle-Nested/README.md`

THE RECIPE

Untangle a nested query

01 Find the innermost brackets: step 1

02 Move them into WITH <name> AS (...)

03 Name the step after one row of it

04 Use the name there; run; compare

QUERY 2 · NESTED

The same step, written twice

Change one copy and forget the other: wrong answer, no warning.

```
SELECT month,
       ROUND(revenue, 2) AS revenue
FROM (SELECT DATE_TRUNC(order_date, MONTH) AS month,
             SUM(order_total) AS revenue
      FROM nakheel.orders
      WHERE status = 'completed'
      GROUP BY month) AS monthly
WHERE revenue > (SELECT AVG(revenue)
                 FROM (SELECT DATE_TRUNC(order_date, MONTH) AS month,
                              SUM(order_total) AS revenue
                       FROM nakheel.orders
                       WHERE status = 'completed'
                       GROUP BY month) AS monthly_again)
ORDER BY month;
```

→ 8 rows

QUERY 2 · ONE CTE, READ TWICE

Written once, read twice

`monthly` in FROM and again inside the subquery.

```
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS month,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY month
)
SELECT month,
       ROUND(revenue, 2) AS revenue
FROM monthly
WHERE revenue > (SELECT AVG(revenue) FROM monthly)
ORDER BY month;
```

→ the same 8 rows · April 2026, 20,237.00

EVERYONE DO Step 3.8

Two branches, one answer

“Which categories grew most from our first year to our second?” Two CTEs that do not need each other, joined at the end.

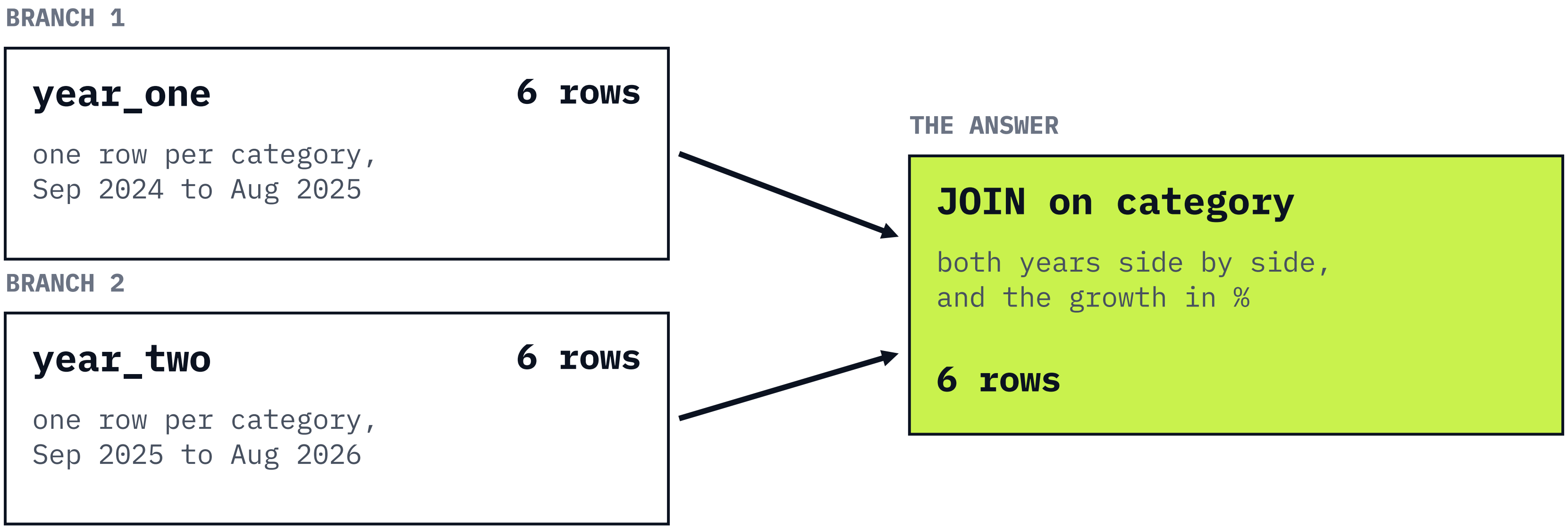
Suggested time · 15 minutes

`labs/D09/05-Evr-Two-Branches/README.md`

PLAN THE STEPS FIRST

Two branches, joined at the end

Each branch starts from the raw tables and is checked on its own. Completed orders only.



BRANCH 1

year_one **6 rows**

one row per category,
Sep 2024 to Aug 2025

BRANCH 2

year_two **6 rows**

one row per category,
Sep 2025 to Aug 2026

THE ANSWER

JOIN on category

both years side by side,
and the growth in %

6 rows

BRANCH 1 · year_one

The 3.5 step, with one new line

BETWEEN keeps both ends: 31 August 2025 is in year 1.

```
WITH year_one AS (
  SELECT p.category,
         SUM(i.qty * i.price) AS revenue
  FROM nakheel.order_items AS i
  INNER JOIN nakheel.orders AS o ON i.order_id = o.order_id
  INNER JOIN nakheel.products AS p ON i.product_id = p.product_id
  WHERE o.status = 'completed'
    AND o.order_date BETWEEN '2024-09-01' AND '2025-08-31'
  GROUP BY p.category
)
SELECT * FROM year_one ORDER BY revenue DESC;
                                       → 6 rows · Outerwear first, 36,318.00
```

THE ANSWER · TWO BRANCHES, ONE JOIN

Join the branches on category

A comma between the two CTEs, none after the last. Then one join.

```
WITH year_one AS ( … year 1 dates … ),
     year_two AS ( … year 2 dates … )
SELECT y1.category,
       ROUND(y1.revenue, 2) AS year_one_revenue,
       ROUND(y2.revenue, 2) AS year_two_revenue,
       ROUND(100 * (y2.revenue - y1.revenue) / y1.revenue, 1) AS growth_pct
FROM year_one AS y1
INNER JOIN year_two AS y2
  ON y1.category = y2.category
ORDER BY growth_pct DESC;

                        → 6 rows · Shoes 37.3 first · Outerwear 11.0 last
```

THE ANSWER · 6 ROWS

Outerwear is still the biggest, and grew least

Every category grew. Check: 142,273.50 + 173,750.00 = 316,023.50, every completed sale.

category	year 1	year 2	growth %
Shoes	26,288.00	36,085.50	**37.3**
Bottoms	22,173.00	28,362.00	27.9
Accessories	19,528.50	23,996.50	22.9
Tops	18,333.00	22,255.00	21.4
Sportswear	19,633.00	22,732.00	15.8
Outerwear	36,318.00	40,319.00	**11.0**

REVIEW

Time's up.
Let's review.

Three shapes of a CTE

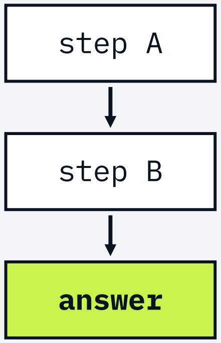
`No Solved file: one drawing`

THREE SHAPES OF A CTE

Chain, reuse, branches

Chain

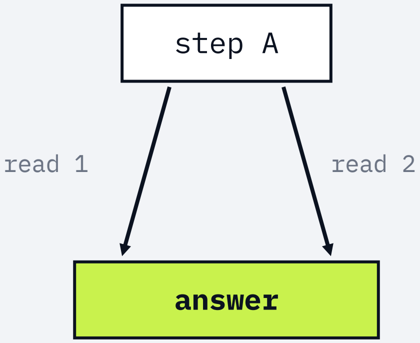
Each step reads the last.


step A
step B
answer

3.2: monthly → average_month

Reuse

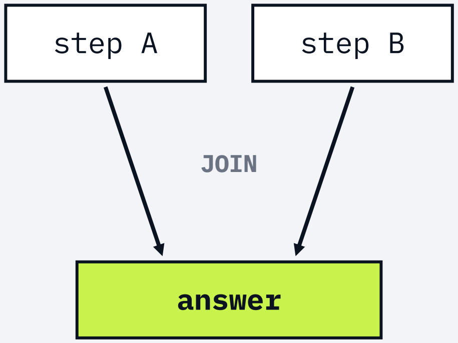
One step, read twice.


step A
read 1
read 2
answer

3.7: monthly, read twice

Branches

Two steps, then a join.


step A
step B
JOIN
answer

3.8: year_one + year_two

Every shape: name each step after one row of it.

Break

Back at 7:00 PM

After the break: save the steps as views for Power BI. Ask in your group channel.

PART 04

Views: save your CTE

A CTE lives while its query runs. A view is a CTE you keep.

INSTRUCTOR DO Step 4.1

Save your CTE as a view

Create the bi dataset, save one query in it, and read it like a table.

Suggested time · 15 minutes

`labs/D08/07-Ins-First-View/README.md`

WHY SAVE IT

A CTE lives for one query

These should use the result by name, without copying the SQL.

Power BI

`reads it from D17`

A colleague

`without your SQL`

Tomorrow’s you

`the same answer, fresh`

DEFINITION

A view is a query saved in the dataset with a name. It stores no rows: every read runs the query again.

● LIVE DEMO

BIGQUERY-SPECIFIC

Create the bi dataset

01 ⋮ next to your project → Create dataset

02 Dataset ID: bi

03 Location: the same as nakheel (Multi-region US)

04 Create dataset

A view must be in the same location as the tables it reads.

SQL · CREATE A VIEW

Save the D06 join as a view

Everything after AS is an ordinary query: the D06 join.

```
CREATE OR REPLACE VIEW bi.fact_sales AS
SELECT i.item_id,
       i.order_id,
       o.order_date,
       o.channel,
       o.status,
       o.customer_id,
       i.product_id,
       i.qty,
       i.price,
       i.qty * i.price AS revenue
FROM nakheel.order_items AS i
INNER JOIN nakheel.orders AS o
  ON i.order_id = o.order_id;
```

→ view created

one row per
order line

every status
kept

A VIEW HOLDS NO ROWS

What you see in the explorer

01 A view icon: fact_sales under bi

02 Schema: its 10 columns

03 Details: the saved SQL

04 No Preview rows and no storage size

READ IT LIKE A TABLE

The audit, from the view

The same numbers as D06.

```
SELECT COUNT(*) AS line_rows,
       COUNT(DISTINCT order_id) AS different_orders,
       ROUND(SUM(revenue), 2) AS all_revenue,
       ROUND(SUM(CASE WHEN status = 'completed' THEN revenue END), 2) AS completed_revenue
FROM bi.fact_sales;



                                             → 7,516 · 3,000 · 340,370.00 · 316,023.50
```

WHAT A VIEW COSTS

0 rows

stored. Every read runs the join again and is billed. On billions of rows, a view everyone reads is a view everyone pays for.

Plan 4.1 · in the sandbox, reads count against the 1 TiB monthly allowance

EVERYONE DO Step 4.2

The customer and product views

One table of events, and small tables that describe it: the D07 cleaning rules, saved once.

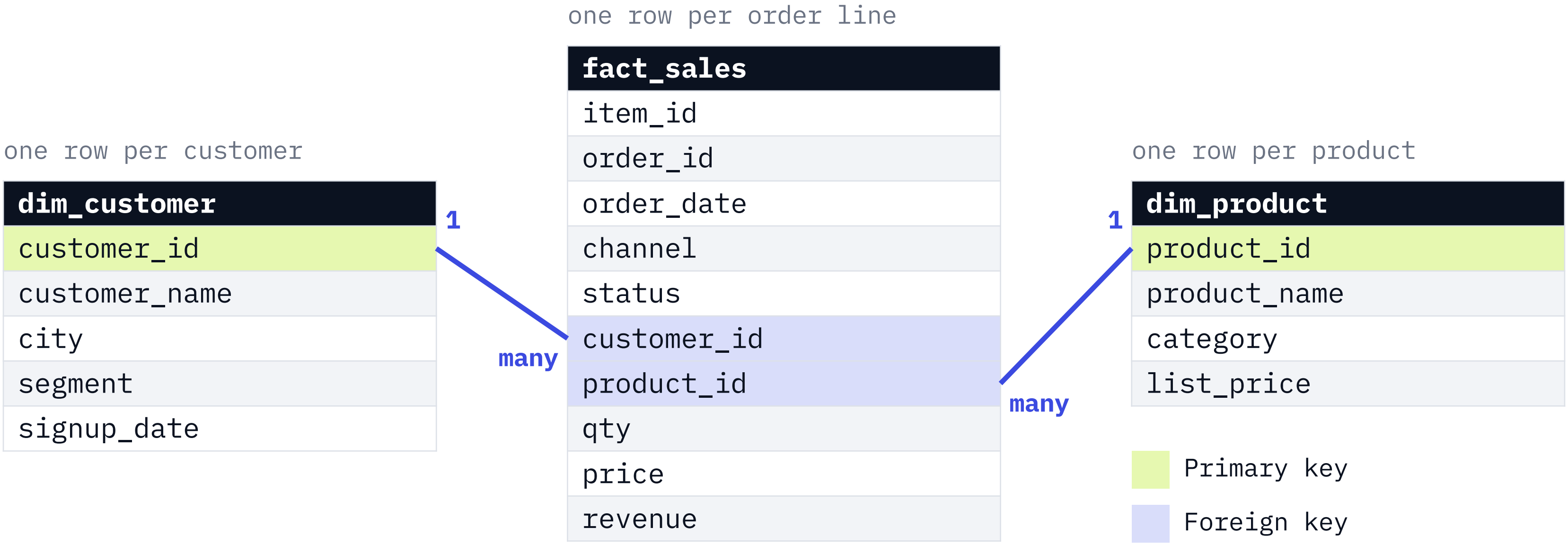
Suggested time · 20 minutes

`labs/D08/08-Evr-Dimension-Views/README.md`

FACT AND DIMENSIONS

fact_sales in the middle, what describes it around

Every report joins fact_sales to these views on the ID.



one row per customer

dim_customer
customer_id
customer_name
city
segment
signup_date

one row per order line

fact_sales
item_id
order_id
order_date
channel
status
customer_id
product_id
qty
price
revenue

one row per product

dim_product
product_id
product_name
category
list_price

🟩 Primary key

🟪 Foreign key

`SQL · bi.dim_customer`

D07’s city rule, saved once

The CASE is D07 3.2’s city rule; the segment line is 3.1’s cleaning.

```
CREATE OR REPLACE VIEW bi.dim_customer AS
SELECT customer_id,
       customer_name,
       CASE
         WHEN city IS NULL THEN 'unknown'
         WHEN LOWER(TRIM(city)) = 'al zarqa' THEN 'zarqa'
         ELSE LOWER(TRIM(city))
       END AS city,
       LOWER(TRIM(segment)) AS segment,
       signup_date
FROM nakheel.customers;
```

CHECK EACH VIEW

Rows equal different IDs

Check 3 of the Five Checks: the ID must be unique here, or every join to fact_sales fans out.

view	rows	different IDs	groups
bi.dim_customer	**500**	**500**	2 segments · 9 cities
bi.dim_product	**60**	**60**	6 categories

REVIEW

Time's up.
Let's review.

CTE or view?

`No Solved file: the comparison, then three quick questions in the chat`

CTE OR VIEW?

Give a step a place to live

	Where it lives	**Lasts**	**Use it for**
CTE	at the top of one query, with a name	while the query runs	a question in several steps, each with its own grain
View	in the dataset, with a name	until you drop it (60 days in the sandbox)	a query other queries, people and Power BI read by name
Subquery	inside one query, in brackets	while the query runs	one number or one list the outer query needs

THREE QUICK QUESTIONS · ANSWER IN THE CHAT

Subquery, CTE or view?

01 The average order value inside a WHERE?

02 Customer totals, then the average per segment, in one report?

03 The cleaned customer list Power BI reads every morning?

INSTRUCTOR DO Step 4.4

Launch the lab

After class: the subquery practice first, then three views.

Suggested time · 7 minutes

`labs/D08/10-Lab-Reporting-Views/README.md`

LAB After class

Reporting views for Power BI and for D11

TASK

Three more views in bi, each with an audit.

FILES

`labs/D08/10-Lab-Reporting-Views/README.md`

`Unsolved/reporting_views.sql`

DONE WHEN

Each view's audit matches the numbers in the README.

After class, after labs/D08/03 and labs/D08/09.

LAB

Lab: Reporting views for Power BI and for D11

In this lab, you will save three more views with an audit each: the customer summary D11 starts from, category by month, and a complete customer list.

When

After class

`labs/D08/10-Lab-Reporting-Views/README.md`

LAB · AFTER CLASS

Lab: Reporting views for Power BI and for D11

CHECKPOINTS	C1 bi.customer_summary: 500 customers, 2,770 orders, 315,085.50, 49 with none. C2 bi.category_month: 144 rows, 316,023.50. C3 bi.dim_customer_complete: 501 rows; INNER JOIN 6,951 lines, 316,023.50.
HINT	C1 is the 3.3 query saved as a view, on top of bi.dim_customer.
TIMING	After class, after labs/D08/03 and labs/D08/09.

Full steps and Check yourself: `labs/D08/10-Lab-Reporting-Views/README.md`

CLOSE

What you can do now

- Put a subquery in WHERE, SELECT or FROM
- Name each step with a CTE
- Join two branches to compare years
- Save a query as a view

NEXT · D10 Months above average in one step; then Assignment 1.